Case study: Low Carbon Index Tracking Portfolio

Antoine ACKERMANN — Emre SARIGOL
Mastère OSE — Mines Paris - PSL & EDHEC
Février 2026

Professeur encadrant : L. Martellini

Summary

A selection of 50 constituents of the S&P500 is performed to achieve a lower carbon intensity while still remaining as representative of the underlying benchmark as possible to ensure reasonable levels of volatility and tracking error. Out-of-sample backtests for five different portfolio strategies using a 2-year calibration window for the estimation of the covariance matrix and a quarterly rebalancing frequency is then carried out on that 50-stock basket. A target reduction of 50% in carbon intensity with respect to the SP500 is achieved on all five portfolios, while still increasing the risk-adjusted performance of the portfolio. Overall, best performance metrics (annualized returns and volatility, Sharpe ratio and tracking error against the SP500) were achieved by the *Max Decorrelation Portfolio*.

Contents

Summary **1**

I. Introduction **2**

II. Methodology **2**

II.1 Data preparation and definitions 2

II.2 Selecting 50 stocks 3

II.3 Optimizing portfolios on the selected universe (low-carbon benchmark construction) 4

II.4 Backtest design 5

II.5 Performance and risk reporting 5

III. Results **6**

III.1 Selection of 50 constituents 6

III.2 Portfolios metrics 6

III.3 Limits and caveats 8

IV. Conclusion: advised strategy and way forward **9**

A Python code for data preparation **10**

B Python code for universe selection **10**

C Python code for portfolios computation **12**

I. Introduction

This case study addresses the construction of a low-carbon SP 500 tracking portfolio for a large US public pension fund currently benchmarked to the cap-weighted SP500. The mandate is twofold: (i) materially reduce the portfolio's carbon impact, proxied by Scope 1 + Scope 2 emissions (via carbon intensity data), and (ii) control benchmark-relative risk, notably the tracking error versus the SP500, while aiming to preserve (or improve) risk-adjusted performance.

To meet these objectives, we implement the two complementary levers required in the assignment: (1) selection (positive screening) of 50 constituents from the SP500 universe, with explicit attention to avoiding excessive sector biases, and (2) portfolio optimization on the selected universe using multiple systematic portfolio construction techniques under explicit carbon-intensity constraints, evaluated through an out-of-sample backtest with a two-year calibration window and quarterly rebalancing.

II. Methodology

II.1 Data preparation and definitions

Market data. We use total return price series for SP500 constituents and the SP500 total return index. All series are resampled to a weekly frequency (forward-filled in case of missing data), and weekly simple returns are computed as percentage changes.

Carbon data. Carbon information is taken as given in the dataset, relying on Scope 1 and Scope 2 emissions and the provided carbon intensity measure. The SP500 "reference" carbon intensity is computed as the cap-weighted average of constituent intensities, using market value weights (benchmark weights). Carbon intensity data has been preferred over carbon footprint data to measure companies' climate impact, since the latter is dependent on (sometimes erratic) market movements that have little to do with a company's ability to generate carbon-efficient economic activity.

Coverage universe. We restrict the investable universe to names for which both (i) carbon data (Scope 1 and 2) are available and (ii) a sufficiently complete price history exists over the analysis window.

Python code for data treatment is given in appendix A.

II.2 Selecting 50 stocks

We devised a mixed-integer linear programming (MILP) to select 50 stocks out of the SP500 under carbon intensity constraints while trying to (a) avoid any sector bias and (b) to cover as much market capitalization of the SP500 as possible. This approach allows to indirectly select the *best-in-class* stocks without falling into the pitfall of selecting stocks only in sector that are inherently low-carbon (e.g. IT, Financials, etc.)

Let $N = \{\text{AAPL, ABBV, ... ZTS}\}$ be the set of individual stocks in the SP500, $S = \{\text{Communication Services, Consumer Discretionary, ..., Utilities}\}$ the set of sectors, CI_i the carbon intensity of an individual stock $i \in N$, CI_{SP500} the SP500 cap-weighted carbon intensity, b_i the weight of i in the SP500, B_s the weight of sector s in the SP500, α the required reduction in carbon intensity for the selection and λ a parameter used in the objective function.

Decision variables

- $z_i \in \{0, 1\}$, equals 1 if the stock is selected,
- $W = \sum_i b_i z_i$ total SP 500 benchmark weight covered by the selected set,
- W_s covered benchmark weight within sector s,
- d_s absolute deviation of sector coverage versus benchmark sector mix.

Constraints

- Cardinality : $\sum_i z_i = 50$,
- Benchmark coverage definition : $W = \sum_i b_i z_i$ and $W_s = \sum_{i \in S} b_i z_i$
- Sector bias : $d_s = |W_s - B_s W|$. This variable will be used to penalize large deviations from the benchmark sector composition.
- Carbon-intensity constraint : $\sum_i b_i CI_i z_i \leq \alpha CI_{SP500}$, with $\alpha = 0.5$ here.

Objectives We maximize benchmark representativeness subject to decarbonization by maximizing covered benchmark weight W while penalizing sector distortions:

$$\max W - \lambda \sum_s d_s$$

with λ calibrated to strongly discourage sector concentration ($\lambda = 100$ displayed satisfying empirical results).

This produces a 50-name subset that (i) covers as much of the benchmark as possible, (ii) remains sector-balanced, and (iii) meets the targeted carbon reduction.

The MILP is implemented in Pyomo and solved using Gurobi with a solver time limit equal to one minute. Python code is given in appendix B.

II.3 Optimizing portfolios on the selected universe (low-carbon benchmark construction)

Given the selected 50-stock universe, we construct alternative long-only portfolios under a carbon-intensity constraint applied directly to portfolio weights to ensure a 50% reduction in carbon intensity, as per the pension fund requests. At each rebalance date, we solve a constrained optimization problem of the form:

Decision variables

- $w_i \geq 0$: portfolio weight of stock i in the selected universe.

Constraints (common to all strategies)

- Carbon intensity limit: $\sum_i w_i CI_i \leq \alpha CI_{SP500}$ with $\alpha = 0.5$
- $\sum_i w_i = 1$

Implemented objective functions (portfolio styles)

1. Global Mean Variance (GMV) :

$$\min_w w^T \Sigma w$$

   where Σ is the sample covariance matrix estimated on the rolling calibration window.

2. Maximum Diversification: we implement a Gurobi-friendly quadratic constrained program by maximizing the weighted sum of marginal volatilities subject to a variance cap, then renormalizing weights to sum to one:

$$\max_w \sum_i w_i \sigma_i$$

3. Maximum Decorrelation :

$$\min_w w^T C w$$

   where C is the sample correlation matrix estimated on the rolling calibration window.

4. Maximum Effective Number of Constituents (dollar contributions proxy). Maximizing effective breadth is equivalent to minimizing weight concentration:

$$\min_w \sum_i w_i^2$$

5. Minimum Tracking Error: let σ^2_{SP500} be SP500 variance and $\sigma_{i,SP500}$ be each asset's covariance with the SP500. Since σ^2_{SP500} is constant, minimizing tracking error is equivalent to:

$$\min_w w^T \Sigma w - 2 \sum_i w_i \sigma_{i,SP500}$$

The risk-parity portfolio is here ignored due to Gurobi's inability to easily solve non-linear problems.

All optimizations are implemented in Python/Pyomo and solved with Gurobi. Python code is given in appendix C.

II.4 Backtest design

We evaluate each strategy using a rolling out-of-sample backtest consistent with the assignment requirements:

- Calibration window: 2 years of weekly returns ($\approx$ 104 observations) to estimate Σ and benchmark covariances.
- Rebalancing frequency: quarterly (implemented as every 13 weeks).
- Between rebalances: weights are held fixed until the next rebalance decision (no re-optimization within a quarter: "buy-and-hold strategy").

II.5 Performance and risk reporting

For each strategy, we report standard institutional diagnostics requested in the case study:

- Annualized return, annualized volatility, and Sharpe ratio (assuming a risk-free rate $r_f = 0$).
- Tracking error versus the SP 500 (annualized standard deviation of active returns).
- Carbon intensity of the portfolio and its reduction versus the SP 500 benchmark.

This framework enables a clear trade-off analysis across the three grading dimensions: decarbonization, risk-adjusted performance, and benchmark tracking quality.

III. Results

III.1 Selection of 50 constituents

The first MILP program allowed us to select these 50 stocks (table 1). Penalizing sector concentrations allowed us to keep sector weights virtually equal to those in the SP500 (fig. 1).

SECTOR NAME	STOCK
Communication Services	COMCAST A, VERIZON COMMUNICATIONS
Consumer Discretionary	HASBRO, AMAZON.COM
Consumer Staples	ESTEE LAUDER COS.'A', COSTCO WHOLESALE, WALMART, PROCTER & GAMBLE
Energy	CHEVRON
Financial	AMERICAN EXPRESS, T ROWE PRICE GROUP, JP MORGAN CHASE & CO., BANK OF AMERICA, WELLS FARGO & CO
Health Care	UNITEDHEALTH GROUP, CVS HEALTH, JOHNSON & JOHNSON, MERCK & COMPANY, PFIZER
IT	APPLE, ADOBE (S), ACCENTURE CLASS A, CISCO SYSTEMS, INTEL
Industrial	PACCAR, BOEING, NORTHROP GRUMMAN, LOCKHEED MARTIN, UNITED TECHNOLOGIES, GENERAL ELECTRIC, CATERPILLAR, HONEYWELL INTL.
Materials	ECOLAB, SHERWIN-WILLIAMS, FMC, PPG INDUSTRIES, BALL, NEWMONT GOLDCORP
Real Estate	PROLOGIS REIT, WELLTOWER, SIMON PROPERTY GROUP, BOSTON PROPERTIES, EQUINIX REIT
Util	EVERSOURCE ENERGY, EDISON INTL., CONSOLIDATED EDISON, SEMPRA EN., EXELON, CMS ENERGY, DOMINION ENERGY

Table 1: 50-stocks selection, ranked by sector.

Selection's carbon intensity is equal to 67.7 gCO_2e per USD of revenue, a 50% reduction from SP500's 135.8 $gCO_2/\$$ of revenue, consistent with the carbon intensity constraints set in the MILP optimization.

III.2 Portfolios metrics

Interestingly, all five tested portfolios all display better returns than benchmark (SP500), despite adding carbon constraints at selection, and again during optimization (fig. 2). These results are obtained thanks to a better diversification of risk, yielding a lower exposure to unrewarded risks.

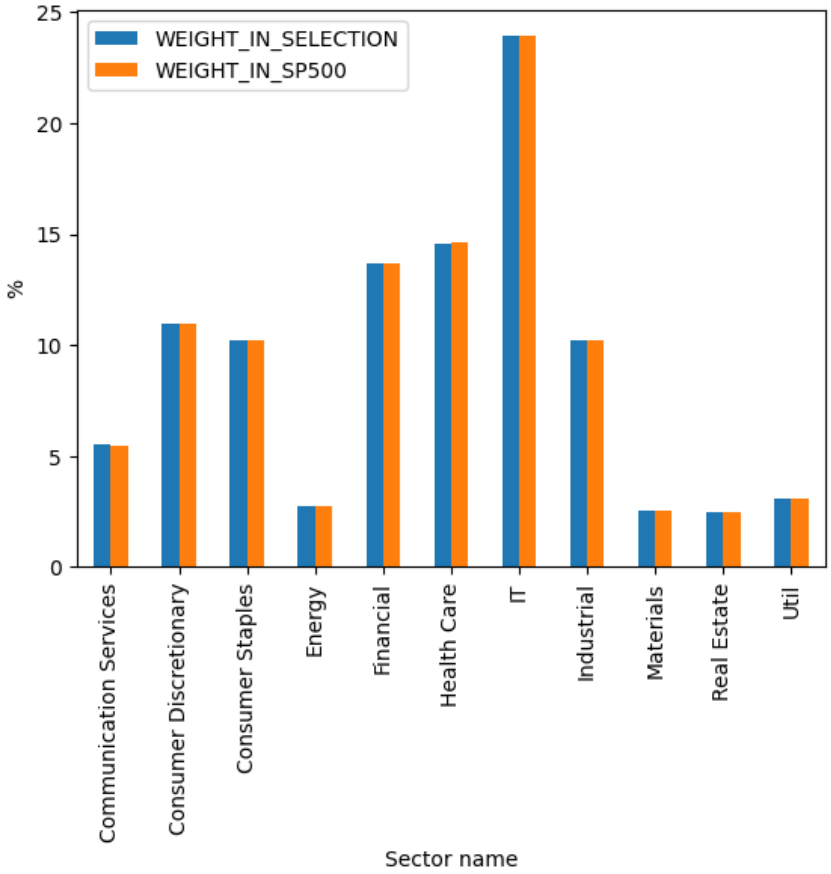

Figure 1: Comparison between sector weights in selection vs. SP500 benchmark.

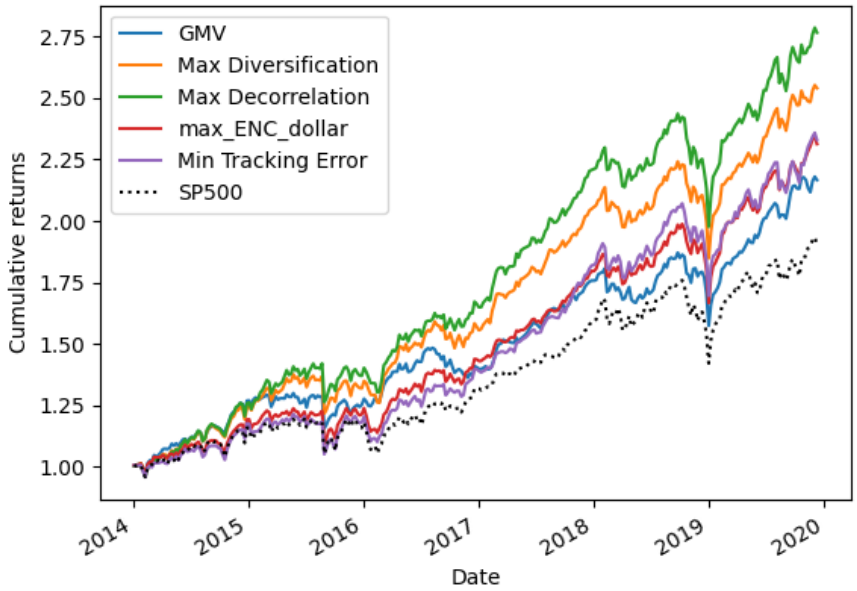

Figure 2: Compounded returns of different portfolios over the backtest horizon.

Across all portfolios, the Max Decorrelation portfolio displays the best risk-adjusted performance (Sharpe Ratio = 1.46) while still remaining at the required 5% tracking error threshold (fig. 3).

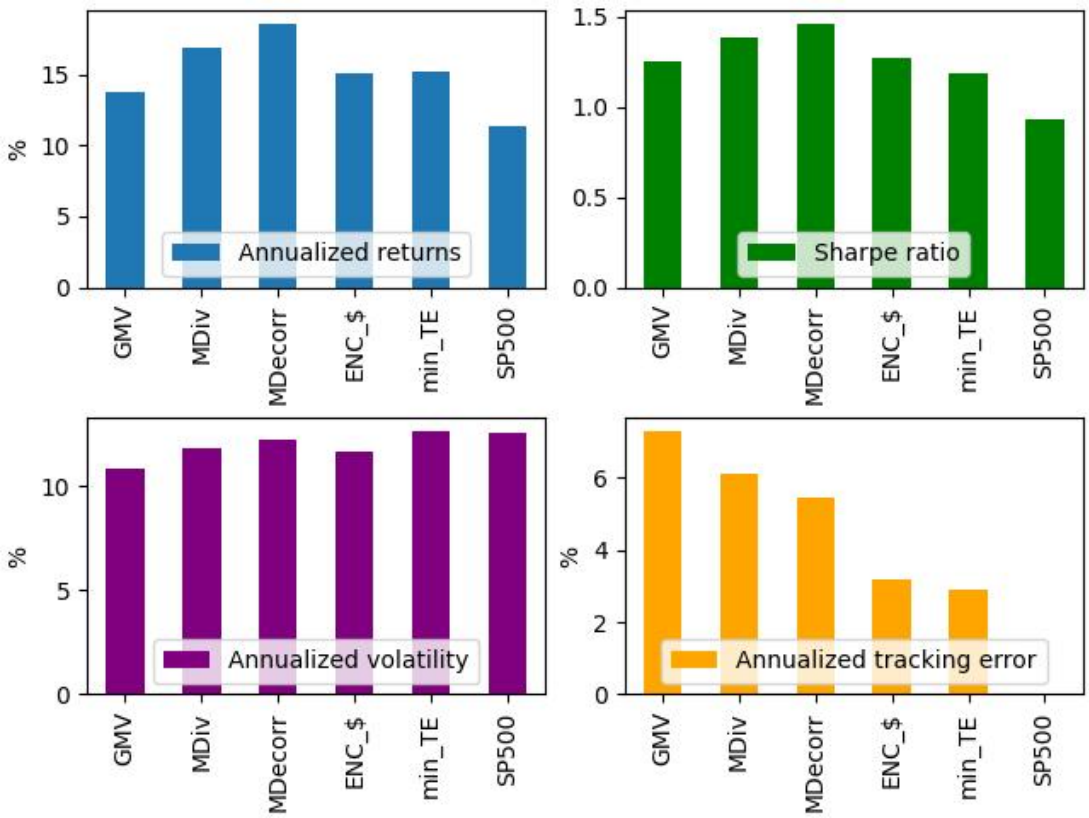

Figure 3: Performance metrics across different portfolios.

III.3 Limits and caveats

Though performed with the best standards of quality in mind, our study suffers from biases and caveats that should be kept in mind by the pension fund's portfolio managers.

Firstly, carbon intensity is considered constant for each stock across the back test time horizon. In practice, carbon data should be updated each year to ensure that *best-in-class* stocks are always selected, thus promoting companies which have made progress in that area and pushing out companies that are falling behind in terms of carbon intensity.

Secondly, when preparing data, we chose to select only stocks for which prices history was complete to us from 2012 to 2020. By doing so, companies that went bankrupt or were delisted from the SP500 during that period (and thus would have offered poor returns) are insidiously removed from our analysis. Likewise, stocks that joined the SP500 during that period were not included in the study. One should therefore be aware of this *survivorship*

bias when analysing the results.

Lastly, a quarterly rebalancing was performed when backtesting portfolios (see fig. 4 for the successive weights of the Maximum Decorrelation Portfolio). There, we assumed that transaction costs were effectively zero. Should there be any such additional fees, the pension fund managers should be aware that they would mar the returns figures shown in section III.

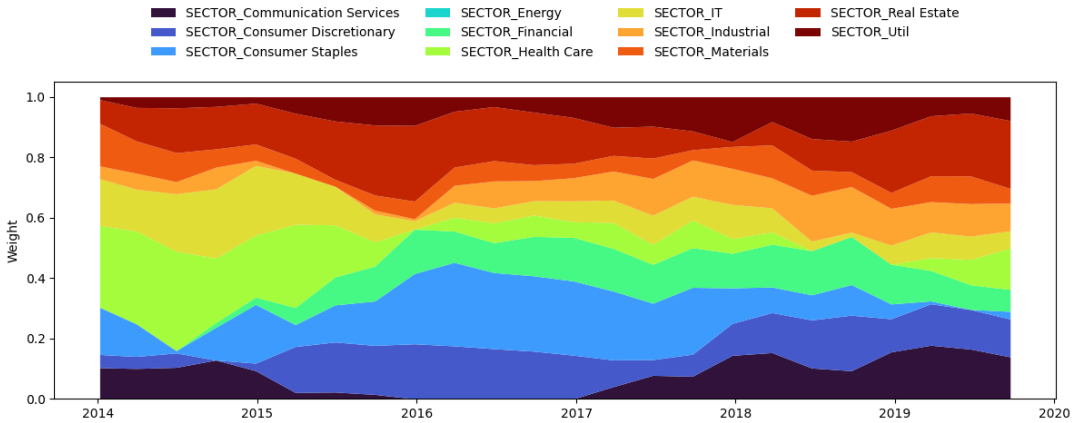

Figure 4: Successive sector weights for the Max Decorrelation Portfolio.

IV. Conclusion: advised strategy and way forward

We followed a two-pronged approach to devise stock portfolios that both offer a better risk-adjusted performance (i.e. a lower exposure to unreward risks) and an improvement in carbon intensity relative to the benchmark SP500 portfolio. In particular, the *Max Decorrelation Portfolio* stands out as offering the best returns while still keeping tracking error against the SP500 below the requested 5% threshold.

We therefore recommend the US public pension fund to pursue this strategy. Selecting the 50 stocks should be done each year with updated carbon intensity data to effectively invest in companies that are able to make environmental progress, measured by their carbon intensity.

A Python code for data preparation

```
import pandas as pd
import numpy as np

stock_prices = pd.read_excel("case study - Individual_stock_data
    .xlsx", sheet_name=0)
stock_prices["Unnamed: 0"] = pd.to_datetime(stock_prices["
    Unnamed: 0"], format="%Y-%m-%d")
stock_prices.set_index("Unnamed: 0", inplace=True)
stock_prices.index.rename("Date", inplace=True)
stock_prices = stock_prices.resample("W").ffill()

carbon_data = pd.read_excel("case study - Individual_stock_data.
    xlsx", sheet_name=1)
carbon_data.rename(columns={"Unnamed: 0": "Code"}, inplace=True)

sp500 = pd.read_excel("case study - Individual_stock_data.xlsx",
     sheet_name=2)
sp500["Name"] = pd.to_datetime(sp500["Name"], format="%Y-%m-%d")
sp500.set_index("Name", inplace=True)
sp500.index.rename("Date", inplace=True)
sp500 = sp500.resample("W").ffill()
```

B Python code for universe selection

```
import pyomo.environ as pyo

alpha = 0.5
lamb = 100
CI_sp500 = (carbon_footprint["INTENSITY"]*carbon_footprint["
    WEIGHT_IN_SP500"]).sum()

m = pyo.ConcreteModel(name="selection 50 in SP500")

m.Stocks = pyo.Set(initialize=carbon_footprint.index.to_list())
m.Sectors = pyo.Set(initialize=by_sector.index.to_list())
```

```
m.CI = pyo.Param(m.Stocks, initialize=carbon_footprint["
    INTENSITY"].to_dict())
m.b = pyo.Param(m.Stocks, initialize=carbon_footprint["
    WEIGHT_IN_SP500"].to_dict())
m.Bs = pyo.Param(m.Sectors, initialize=by_sector["
    WEIGHT_IN_SP500"].to_dict())
m.CI_sp500 = pyo.Param(initialize=float(CI_sp500))
m.alpha = pyo.Param(initialize=float(alpha))
m.lamb = pyo.Param(initialize=float(lamb))

sector = carbon_footprint["SECTOR_NAME"].to_dict()
m.I_by_S = pyo.Set(
    m.Sectors,
    initialize=lambda mm, s: [i for i in mm.Stocks if sector[i]
        == s],
    ordered=False
)


# VARIABLES
m.z  = pyo.Var(m.Stocks, within=pyo.Binary)
m.W  = pyo.Var(within=pyo.NonNegativeReals)
m.Ws = pyo.Var(m.Sectors, within=pyo.NonNegativeReals)
m.d  = pyo.Var(m.Sectors, within=pyo.NonNegativeReals)


m.card = pyo.Constraint(expr=sum(m.z[i] for i in m.Stocks) ==
    50)


m.defW = pyo.Constraint(expr=m.W == sum(m.b[i] * m.z[i] for i in
    m.Stocks))


def defWs(mm, s):
    return mm.Ws[s] == sum(mm.b[i] * mm.z[i] for i in mm.I_by_S[
        s])
```

```
m.defWs = pyo.Constraint(m.Sectors, rule=defWs)


def _dev_pos(mm, s):
    return mm.d[s] >= mm.Ws[s] - mm.Bs[s] * mm.W
def _dev_neg(mm, s):
    return mm.d[s] >= -(mm.Ws[s] - mm.Bs[s] * mm.W)

m.dev_pos = pyo.Constraint(m.Sectors, rule=_dev_pos)
m.dev_neg = pyo.Constraint(m.Sectors, rule=_dev_neg)

m.carbon = pyo.Constraint(
    expr=sum(m.b[i] * m.CI[i] * m.z[i] for i in m.Stocks) <= m.
        alpha * m.CI_sp500 * m.W
)

m.obj = pyo.Objective(
    expr=m.W - m.lamb * sum(m.d[s] for s in m.Sectors),
    sense=pyo.maximize
)


optimizer = pyo.SolverFactory("gurobi")
optimizer.options["TimeLimit"] = 60
optimizer.solve(m)
```

C Python code for portfolios computation

```
CI_sp500 = (carbon_footprint["INTENSITY"]*carbon_footprint["
    WEIGHT_IN_SP500"]).sum()

def compute_weights(returns, strategy, sp500_returns,
    carbon_footprint=carbon_footprint_selec, CI_sp500=CI_sp500,
    alpha=0.5):

    m = pyo.ConcreteModel(name="portfolio optimization")
```

```
# Sets
m.N = pyo.Set(initialize=list(returns.columns), ordered=True
    )

#Parameters
n_stocks = len(returns.columns)
m.n_stocks = pyo.Param(initialize=n_stocks)
m.CI = pyo.Param(m.N, initialize=carbon_footprint["INTENSITY
    "].to_dict())
m.CI_sp500 = pyo.Param(initialize=CI_sp500)
m.alpha = pyo.Param(initialize=alpha)

#Variables
m.w = pyo.Var(m.N, within=pyo.NonNegativeReals)

#Constraints
m.sum_weights = pyo.Constraint(rule=lambda m: sum(m.w[i] for
    i in m.N) == 1)
m.carbon = pyo.Constraint(expr=sum(m.w[i] * m.CI[i] for i in
    m.N) <= m.alpha * m.CI_sp500)

#Objective function : depends on portfolio type

match strategy:
    case "GMV":
        Sigma = returns.cov()
        m.Sigma = pyo.Param(m.N, m.N, initialize=Sigma.stack
            ().to_dict())
        def GMV(m):
            return sum(m.w[i]*m.w[j]*m.Sigma[i, j] for i in
                m.N for j in m.N)

        m.obj = pyo.Objective(rule=GMV, sense=pyo.minimize)

    case "Max Diversification":
        Sigma = returns.cov()
        sigma = returns.std()
        m.Sigma = pyo.Param(m.N, m.N, initialize=Sigma.stack
```

```
        ().to_dict())
    m.sigma = pyo.Param(m.N, initialize=sigma.to_dict())

    def MaxDiv(m):
        return sum(m.w[i]*m.sigma[i] for i in m.N)

    m.del_component(m.sum_weights)
    m.del_component(m.carbon)

    m.var_cap = pyo.Constraint(expr=sum(m.w[i] * m.w[j]
        * m.Sigma[i, j] for i in m.N for j in m.N) <=
        1.0)
    m.carbon = pyo.Constraint(expr=sum(m.w[i] * m.CI[i]
        for i in m.N) <= m.alpha * m.CI_sp500 * sum(m.w[i
        ] for i in m.N))
    m.obj = pyo.Objective(rule=MaxDiv,sense=pyo.maximize
        )


case "Max Decorrelation":
    Corr = returns.corr()
    m.Corr = pyo.Param(m.N, m.N, initialize=Corr.stack()
        .to_dict())
    def MaxDecorrel(m):
        return sum(m.w[i] * m.w[j] * m.Corr[i, j] for i
            in m.N for j in m.N)

    m.obj = pyo.Objective(rule=MaxDecorrel, sense=pyo.
        minimize)

case "max_ENC_dollar":
    def max_ENC_dollar(m):
        return sum(m.w[i]**2 for i in m.N)

    m.obj = pyo.Objective(rule=max_ENC_dollar, sense=pyo
        .minimize)


case "Min Tracking Error":
```

```
            Sigma = returns.cov()
            Cov_i_SP500 = returns.apply(lambda s: s.cov(
                sp500_returns["S&P 500 COMPOSITE - TOT RETURN IND
                "]))

            m.Sigma = pyo.Param(m.N, m.N, initialize=Sigma.stack
                ().to_dict())
            m.Cov_i_SP500 = pyo.Param(m.N, initialize=
                Cov_i_SP500.to_dict())

            def MinTE(m):
                return sum(m.w[i]*m.w[j]*m.Sigma[i, j] for i in
                    m.N for j in m.N) - 2*sum(m.w[i]*m.
                    Cov_i_SP500[i] for i in m.N)
            m.obj = pyo.Objective(rule=MinTE, sense=pyo.minimize
                )

    optimizer = pyo.SolverFactory("gurobi")
    optimizer.options["TimeLimit"] = 60
    optimizer.solve(m)

    w = {i: pyo.value(m.w[i]) for i in list(m.N)}
    w = pd.Series(w)

    if strategy == "Max Diversification":
        # Renormalisation only for MaxDiv
        s = w.sum()
        if s > 0:
            w = w / s

    return w

def backtest_rolling(returns, strategy, sp500_returns=
    sp500_returns, window=2):
    T = returns.index.get_loc(returns.index.max())
    t0 = window*52+1
    portfolio = {}

    R_OOS = []
```

```
    dates_OOS = []

    for t in range(t0, T): # need at least t0 data points to
        begin the calculation
        R_insample = returns.iloc[t-window*52:t].copy()
        R_sp500 = sp500_returns.loc[R_insample.index].copy()

        if t%13 == 0 or t==t0: #quarterly rebalancing
            w = compute_weights(R_insample, strategy, R_sp500)
            portfolio[returns.index[t]] = w

        R_OOS.append(w.T @ returns.iloc[t])
        dates_OOS.append(returns.index[t])


    return pd.Series(R_OOS, index=dates_OOS, name=strategy), pd.
        DataFrame.from_dict(portfolio, orient="index")

strategies = ["GMV", "Max Diversification", "Max Decorrelation",
    "max_ENC_dollar", "Min Tracking Error"]
R_OOS = {}
portfolios = {}

for strat in strategies:
    print(strat)
    R_OOS[strat], portfolios[strat] = backtest_rolling(
        stock_returns, strat)

R_OOS = pd.DataFrame(R_OOS)
```